

Earning Autonomy: Learning and Applying an Organizer's Decision Policies in Simulated Group Coordination

Joshua H. Kung · Department of Computer Science, Columbia University · jhk2214@columbia.edu

Advisor: Lydia B. Chilton, Computational Design Lab, Columbia University

Final Report: Distributed REsearch Apprenticeships for Master's (DREAM), CRA-WP, Summer 2026

Abstract

Agentic systems can execute tasks competently but cannot yet execute them the way a particular person would. We present a system that captures an individual organizer's coordination decisions, together with the reasons behind them, as explicit, inspectable policies of the form condition → action → rationale, graduates each policy along an autonomy ladder from proposed to confirming to autonomous, suspends any policy the user overrides and records the correction as a rule that binds subsequent drafts, and escalates situations that are novel or that contradict an existing policy rather than generalizing silently from a single event. All task-specific vocabulary lives in a thin adapter; the engine that ranks, escalates, and renders for the user never sees this. We exercise it in a multi-agent simulation testbed built on scheduled agents operating over a shared state file, in which simulated participants hold private motives withheld from the organizer's view, misreport their intentions, hold out for better offers, and renege late. We instantiate the system on running an intramural basketball team, organized by the author, across a full simulated season: first as a 3v3 team, then stress-tested with a 5v5 roster. We develop four design arguments: that a decision interface earns its value only in the hard region of a decision, so information density should track local difficulty rather than spread uniformly; that when the system's own ranking metric cannot separate the candidates at the boundary, stating the indifference is more useful than manufacturing a recommendation; that composition constraints make candidate value marginal-given-need rather than absolute; and that domain-generalizability is a testable property, which we test by relabeling the interface to unrelated coordination tasks through an adapter swap alone. The escalation-to-policy loop runs end-to-end and produced four tracked policies across the season. This report is deliberately scoped as a system and design contribution: it builds the engine and testbed that a user evaluation requires and develops the arguments such an evaluation would test, but it does not include one, and we are explicit about what that prevents us from claiming.

1. Introduction

AI agents have become competent at execution. Given a clear instruction, a modern agent will send the message, book the slot, or fill the form. What remains difficult is doing these things the way a particular person would have done them, and the gap is widest in the small recurring decisions that fill a week: who to put on a project, who to ask first, whether to push a meeting or drop the person who keeps missing it. These are not tasks where the instruction is the substance. The judgment is the substance, and delegating them requires the agent to know not only what the user would do but why.

This project takes up a narrower question than whether agents can act autonomously: can an agent learn a specific person's decision-making well enough to earn the right to act for them incrementally,

auditably, and without quietly going wrong? We treat this as optimization over known constraints, perturbed, as real coordination is, by disruptions that no one planned for.

Three commitments follow from taking that question seriously.

First, learned rules must carry their rationale, not just their action. The same observed choice admits many explanations (the organizer picked the strongest candidate because they value raw quality, or because they needed a specific capability, or because that person apologized last week), and a rule learned with the wrong reason will be confidently wrong the first time the situations diverge.

Second, autonomy must be earned per-decision rather than granted globally. A rule that appears once has not demonstrated anything. It should climb a ladder as it is confirmed, and when the user overrides it, the correction should bind the agent from that point on.

Third, and least obviously: the mechanism that decides when to act independently and the mechanism that decides when to interrupt the human are the same mechanism. We initially treated these as opposed, since one implies asking less and the other asking more; they are not. A detector for situations that are novel or that contradict existing rules is exactly the gate governing when autonomy should escalate. An agent without it is not more autonomous; rather, it is just less careful.

Building the interface for such a system surfaced a fourth question we had not anticipated, and which occupied the second half of the project. Given a ranking the agent has produced, where in the decision should the interface actually do work? The default answer, implicit in most decision-support UIs, is everywhere: render all candidates with all attributes and let the user sort it out. We argue this is wrong in a specific and correctable way, and Section 6 develops the alternative.

Contributions

1. An engine that captures decision policies together with their rationale and graduates their autonomy, separated from task vocabulary by an adapter seam whose rendering path passes a relabeling test (Sections 4, 6.4).
2. A multi-agent simulation testbed in which participants have private motives, misreport, and renege. These are the conditions under which naive automation fails (Section 5).
3. The hard-region argument: a decision interface should allocate information density in proportion to local decision difficulty, and should decline to recommend when its own metric is indifferent (Sections 6.1, 6.2).
4. Constraints-as-shape: when a selection must satisfy composition requirements, candidate value becomes marginal-given-need, and constraint relaxation should be escalated rather than performed silently (Section 6.3).

Scope. This report makes a system and design contribution, not an empirical one, and that choice is deliberate. A study of whether learned policies match a person's judgment needs two things that did not exist at the start of the summer: an engine that captures a decision together with its reason, and a testbed in which decisions are hard enough for the difference between good and poor policies to show. This project builds both and uses them to surface the design questions such a study would test. The organizer throughout is the author, who runs the team and made every call the system escalated; Section 3 describes this arrangement and Section 10 describes the evaluation it prepares for. We are explicit throughout about the boundary between what runs, what is implemented but not integrated, and what remains unbuilt. Section 9 states this plainly.

2. Related Work

2.1. Mixed-initiative interaction and the timing of interruption

The question of how automation and direct manipulation should share control is long-standing. Horvitz's principles of mixed-initiative user interfaces [1] argue that an intelligent system should reason about the expected value of acting, asking, or doing nothing, and should time its interventions rather than assume more automation is better. The LookOut scheduling system made this concrete: infer the user's intent, and open a dialog when inference is uncertain. Our escalation gate (Section 4.3) inherits this framing directly.

Our cut-line design narrows it in a specific way. Horvitz asks when a system should intervene in an ongoing task; we ask *where within a single decision* the interface should spend its budget, and argue that the answer is the contested boundary rather than the decision as a whole. Horvitz and Barry's expected value of displayed information [2], which asks what a display should show a decision-maker whose attention is limited, is the closest antecedent to what we call difficulty-driven information density, and Kocielnik et al. [3] supply the empirical counterpart: how a system sets expectations about its own competence changes acceptance independently of that competence. A board that renders every candidate with equal confidence is making an expectation-setting claim whether or not it intends to.

2.2. Learning decision rules from observed behavior

Our central technical move, capturing a person's decision policy from the choices they make, sits in a literature with a long history and a known central difficulty.

Inverse reinforcement learning [4] formalizes the problem of recovering the objective a demonstrator was optimizing, and establishes at the outset that the problem is ill-posed: many reward functions explain the same observed behavior. Apprenticeship learning [5] addresses this by matching feature expectations rather than recovering a unique reward, and maximum-entropy IRL [6] resolves the ambiguity by preferring the least-committal distribution consistent with the demonstrations. Preference-based approaches [7] sidestep reward specification entirely by learning from comparative human judgments. Argall et al. [8] survey the broader learning-from-demonstration setting.

This literature names the problem we hit in Section 8 and gives it a sharper statement than we arrived at independently. Our observation that “the same observed choice admits many explanations” is reward ambiguity. Our failure mode (a rule that learns the organizer's action while attaching the wrong reason, then generalizes confidently in the wrong direction) is the causal-confusion result of de Haan et al. [9]: an imitator that latches onto a correlate of the expert's action rather than its cause performs worse with more information, not better.

We depart from this line in how the reason is obtained and how it is represented. IRL infers a reward function from behavior alone, and that function is compact, transferable, and uninspectable. We do not infer the reason; we ask for it. At each escalation the organizer states why they made the call, a language model drafts a condition and rationale from that statement and the situation, and the organizer confirms or edits the draft before it is stored. The result is an explicit rule, which is verbose, brittle outside its condition, and, critically, presentable to the user for confirmation or rejection. The tradeoff is deliberate: our user cannot audit a learned reward vector, but they can read “when the roster is short at a position and the candidates are close in value, prefer the dependable one” and tell us it is wrong. Eliciting and confirming the rationale, rather than inferring it from the action, is our answer to reward ambiguity, and it is available only because the representation is symbolic and the demonstrator is present to be asked. What remains learned is generalization: whether a new situation falls under a stored condition is judged by the model, not restated by the organizer. We do not claim a

better learner; we claim a representation whose errors are legible to the person whose judgment is being modeled.

The same commitment places us within programming by demonstration [10, 11], where a system generalizes a procedure from watching a user act, and specifically within the line of personal assistants that learn an individual's preferences from their own behavior. Mitchell et al.'s calendar apprentice [12] learned a particular user's scheduling preferences from their observed decisions and is the closest historical statement of our thesis; PTIME [13] and CMRadar [14] continued it with richer preference models for calendaring. What distinguishes our setting is not the learning but what is being learned: these systems model preferences over outcomes in a domain with a well-specified objective, whereas our decisions are contested at the margin and have no ground truth apart from the organizer's own judgment. This is why the interface must be willing to say it cannot separate two options (Section 6.2), a move that has no counterpart in a system that believes its objective.

2.3. Graduated and adjustable autonomy

That autonomy should be held in degrees rather than granted wholesale is an established position. Sheridan and Verplank's levels of automation [15] grade automation by degree, and Parasuraman, Sheridan and Wickens's later model [16] adds stage (information acquisition, analysis, decision selection, and action implementation), arguing that a system may sit at different levels in different stages. Adjustable-autonomy work in multi-agent systems [17] makes the level itself dynamic, transferring control between agent and human as the situation warrants, and Goodrich and Schultz [18] survey how the same question is posed in human-robot interaction.

Our autonomy ladder is an instance of this, with two differences worth stating plainly. First, the level is held **per policy** rather than per system or per stage: the same agent acts autonomously on one recurring decision while asking on another, and the unit of autonomy is a learned rule rather than a functional stage. Second, the level moves as a function of the agent's own track record on that rule, with promotion on repeated confirmation and, on override, suspension of the rule and a binding record of the user's correction, rather than being set by an operator or by a policy for transferring control under load. The adjustable-autonomy literature largely asks who should hold control *now*; we ask what an agent must have demonstrated *about a specific rule* before it holds control at all.

2.4. Trust calibration and the decision not to recommend

Whether people accept an agent that acts for them depends less on capability than on calibrated reliance. Lee and See [19] distinguish misuse from disuse and argue that appropriate reliance requires a system to make its purpose, process, and performance legible. Bansal et al. [20] sharpen this empirically: explanations increased the rate at which people accepted the AI's recommendation whether or not it was correct, and did not improve team performance beyond simply displaying the AI's confidence, because the reliance they induce is not discriminating. Miller [21] argues that explanations are contrastive and social rather than causally complete, which is why our rationale field stores *why this rather than that* rather than a trace; Lim and Dey [22] show that demand for intelligibility is situational, which is the empirical form of our claim that information density should track difficulty.

These motivate two design commitments. The autonomy ladder exists so that reliance is built incrementally against observed behavior rather than asserted up front. And the refusal to fake a recommendation (Section 6.2) follows from the same logic: a system that always renders confident output makes a strong and a coin-flip recommendation indistinguishable, which is precisely the indiscriminating reliance Bansal et al. identify.

The formal counterpart is the reject option in classification. Chow [23] established the optimal tradeoff between error and abstention, and the selective-prediction literature [24] develops the conditions under

which a predictor should decline to predict. Our cut-line behavior is this idea moved into an interface: when the value metric cannot separate the two candidates straddling the boundary, the system abstains from ranking them and renders the tradeoff instead. We differ from the classification setting in what abstention costs. A selective classifier that abstains defers to nothing in particular; our abstention hands the decision to a person who is the actual authority on it, and is designed to treat their resulting choice as a rule worth learning (the write-back is not yet implemented; Section 9). In this design, abstention is not a failure to answer but the point at which the system acquires information it could not have derived.

2.5. Agentic coordination with human oversight

DoubleAgents [25], developed in the same lab, is the immediate predecessor to this work and shares its setting. It integrates a coordination agent that maintains state and proposes plans and actions, a dashboard visualization that makes the agent's reasoning legible for the user to evaluate, and a policy module that transforms a user's edits into reusable alignment artifacts (coordination policies, email templates, and stop hooks) so that behavior improves over repeated use. Earlier versions of the paper also describe a built-in respondent simulation that lets users rehearse and refine policies before live use. Across a lab study and real-world deployments it reports that comfort with offloading tasks and reliance on the system grew over time, while users continued to require control at points of uncertainty.

We share its architecture and its simulation-first method, and differ in three specific ways that define this project's scope. **Origin:** DoubleAgents' policies derive from user edits and authored artifacts: the user states what they want, and the system captures it. Ours are captured at the moment of decision. When the system escalates, the organizer makes the call they had to make anyway and states the reason for it; the system drafts a condition from that reason, the organizer confirms it, and the choice and the rationale are stored together as a policy for that decision point. What is learned is a preference together with the reason it is held, which is what the rationale problem of Section 2.2 requires. **Status:** our policies carry an autonomy level that changes over time as a function of confirmation and override, so a rule's standing is a running claim about the agent's demonstrated reliability rather than a fixed property. **Scope:** DoubleAgents is built around a particular coordination workflow, seminar scheduling in its deployments, and evaluates that workflow end to end. Our engine is situation-agnostic by design: everything task-specific lives in an adapter (Section 4.1), and the same rendering code serves basketball, office-hours staffing, and dinner coordination (Section 6.4). Within that, we examine a single constrained-selection decision in depth, which is what surfaces the interface findings in Section 6 and would be invisible at workflow granularity.

Two adjacent systems bound the design space from either side. Zoro [26] gives users active rules that an agent must comply with during code generation, making a standing ruleset the control surface; Sreedhar et al. [27] let users author workflow behaviors conversationally in a customizable email system, and report that user-authored functionality introduces mis-specification that users must manage through ongoing oversight and refinement. Both take the rules as *given by the user* and ask how to enforce and maintain them. We take the rules as *drafted from the user's decisions* and ask what the agent must demonstrate before acting on them. The mis-specification risk Sreedhar et al. observe is strictly worse in our setting, since a condition drafted by a model from a single stated reason can be wrong in ways the user has not considered, which is the argument for making confirmation of the rationale mandatory before a policy leaves proposed.

2.6. Simulation as a design testbed

Park et al. [28] demonstrated that LLM-driven agents in a persistent environment produce believable and partly emergent social behavior, and earlier work on populated prototypes [29] established simulation as a way to surface edge cases before deployment. Within our lab, Sreedhar and Chilton [30] compare single-model prompting with multi-agent configurations in the ultimatum game and find that

the multi-agent configuration tracks human behavior more closely, and Sreedhar et al. [31] show that multi-agent systems reproduce human prosocial behavior from public goods game experiments. AgentDynEx [32] addresses the control problem such simulations raise: how a designer nudges a multi-agent simulation toward the conditions they want to study without scripting away the emergence that makes it worth running.

We adopt this method for a narrower purpose. Our simulated participants exist not to be believable but to make the decision hard in the specific ways real coordination is hard: private motives, unreliable commitment, and reneging. Section 8 argues that decision fidelity, not behavioral realism, is the appropriate target for a testbed of this kind: a simulation unrealistic in ways that do not affect the decision is acceptable, while one that is lifelike but decision-trivial is not. This inverts the usual evaluation criterion for social simulation, and we take the AgentDynEx framing as support: if a simulation must be steered toward the conditions of interest, then the conditions of interest, not the realism, are what the designer is actually specifying.

3. The Coordination Setting

We needed a task where judgment is genuinely required and where naive automation would visibly fail. Everyday group coordination (assembling a team, staffing a shift, filling a table) has four properties that make it well suited, and the properties generalize past any particular instance.

Preferences are private. Participants do not state why they are actually in or out. The organizer infers it from behavior over time, and the behavior can be strategic.

Commitment is unreliable. A “yes” is not a promise. People over-commit, stall, go quiet, and renege close to the event. A system that treats an RSVP as ground truth is modeling a different problem.

There is no infeasible state. The event happens regardless; the organizer never fails outright, only does better or worse. This makes coordination optimization-flavored rather than constraint-satisfaction-flavored, which means the system must judge rather than solve.

Participation is endogenous. Allocation decisions change who is willing to participate next time. Choosing the strongest available option today makes the passed-over candidates less likely to be available tomorrow. The decision and the decision environment are coupled.

Any coordinator stands in this structure: a manager staffing shifts, an instructor assigning office hours, a friend organizing a game. We instantiated the system across several such scenarios during development (a lab game night, a watch party with cost and travel constraints, a camping trip, office-hours staffing) before settling on running a 3v3 intramural basketball team as the primary instance, because its composition constraints are crisp and its reliability signal is legible across repeated events. We then stress-tested the system in two directions: a larger 5v5 roster, which is the scenario Sections 6.1 and 6.3 use, and planning where a group should go to dinner (Section 6.4). The large majority of the work went into the basketball instances.

The organizer in this instance is the author, who runs the team. Each time the system escalated a decision, the author made the call and stated the reason, and the system recorded both. This is autobiographical design in the sense of Neustaedter and Sengers [33]: the designer is also a genuine user of the system, which keeps the design in close contact with a real need at the cost of generality. Every policy reported in Section 7 is therefore one person’s judgment, and nothing in this report shows that the same ladder or interface serves other organizers.

The fourth property is the one this work engages least, and we return to it in Sections 9 and 10.

4. System

4.1. Overview

The system separates a domain-agnostic engine from a thin per-task adapter. The engine handles policy representation, evaluation, escalation, and rendering, and never encounters domain vocabulary. The adapter holds everything task-specific: decision points, feature extraction, choice classification, and display labels. Core rendering runs on the Python standard library with no external dependencies.

4.2. Policy representation and the autonomy ladder

Learned rules take the form:

```
decision_point + condition → action + rationale
```

The rationale is stored as a first-class field rather than as a comment. This is the central design decision of the learning component. When the system proposes a rule back to the user for confirmation, the user is confirming the reason, not the move, which is what makes the rule generalize correctly to situations the original observation did not cover.

When the organizer resolves an escalation, a language model drafts the condition and rationale from the situation and the organizer's stated reason, and the organizer confirms or edits the draft before it is stored. At run time the same model judges whether a new situation satisfies a stored condition, using the features the adapter extracts.

Each policy carries an autonomy status and counters:

Status	Meaning
flagged	Surfaced for review; not yet a rule
proposed	Drafted from an escalated decision, awaiting first confirmation
confirming	Confirmed at least once; still asks the organizer to decide
auto	No longer asks for the decision; drafts the action with its stored rationale as a suggestion
paused	Suspended after override or audit failure

Promotion follows repeated confirmation: once the organizer has made the same call for the same reason twice, the third occurrence is handled as a suggestion, drafted with the stored rationale, rather than as a question. A user override suspends the policy and records the organizer's correction in the list of rules the agent must follow; every subsequent draft and decision is checked against that list.

Autonomy is therefore held per-decision-point rather than granted to the agent as a whole, and it is always reversible by the ordinary act of disagreeing with the system. Because every outbound message passes through the approval step described in Section 5.1, even an auto policy proposes rather than sends; what it earns is the right to stop asking the organizer what to do.

A case from the season shows the ladder at work. A player said they would not attend unless they were guaranteed playing time. No policy covered this, so the system escalated. The organizer promised the player some playing time and gave the reason: the team needed at least four players including the

organizer, refusing risked the player's attendance at future games, and the player was a reasonably strong contributor. The system stored the choice and that rationale together as a policy. When the same kind of demand recurred and the organizer made the same call twice, the policy was promoted, and from the third occurrence on the system suggested the promise with the organizer's own reasons attached. The same thread later produced the season's clearest case of a faithful policy backfiring, which Sections 7 and 8 return to.

4.3. Escalation

Before a policy is applied, a gate asks whether the current situation is new or unclear: an unfamiliar decision point, one that contradicts a rule already held, or one where held policies conflict or none gives a clear way forward. If so, the system stops, flags, and asks, rather than appending a policy from a single event. The user's resulting decision becomes a tracked policy entering the ladder at proposed.

This is the behavior the project is organized around, and it unifies the two goals that initially seemed opposed. Escalation is not an exception to autonomy; it is the mechanism that governs it. The system acts on the routine and raises a hand on the unfamiliar.

A companion reflection pass audits new and changed policies, returns a proceed-or-hold verdict before action, and records intended-versus-applied behavior afterward, so that divergence between what a policy specified and what was executed is visible rather than silent.

4.4. Persistence

Three stores hold durable knowledge, scoped so that domain-specific facts stay local while general rules can be promoted:

- **Policies:** captured rules with rationale, ladder status, and confirmation/override counts.
- **Learnings:** durable facts and directives (for example, who owns equipment), with confidence and provenance.
- **Metrics:** proposed measurements that would verify whether a policy is working, together with recurring themes and dependencies mined from history.

Current-world state lives in a single JSON object: participants and their attributes, the candidate pool, a four-state RSVP field (in / maybe / out / silent), observed reliability as show and flake counts, composition rules, event history, and the message log. The four-state RSVP model matters: a non-reply is not a decline, and the system never converts one into the other.

Two fields exist specifically to make the simulation non-trivial. Hidden motives carry ground-truth attendance and bench-aversion traits that are withheld from the organizer's view, so the organizer's information is genuinely incomplete. Scripted conditions inject deterministic scenario events such as double bookings.

5. Simulation Testbed

5.1. Architecture

The testbed runs as a set of scheduled agents operating over a shared JSON state file. No real messaging occurs; the state file is the entire world.

The coordinator agent runs on a fixed interval. Each cycle it reads messages arriving since its cursor, evaluates applicable policies, and may draft at most one outbound message per run. Critically, a draft is written to a `pending_draft` field rather than appended to the message log, and every draft carries the

policies that triggered it with their identifiers, labels, and reasons. Confirmation is chat-based: the draft is printed for the organizer to approve, skip, or edit. While a draft is pending, the coordinator will not produce another.

This structure is the boundary between proposing and acting, made mechanical. The agent cannot take a social action without passing through a point where the human can see the action and the reasoning that produced it.

Persona agents, one per simulated participant, run on a shorter interval. Each reads the state file, finds unread messages addressed to it, rolls a reply probability, responds in character, appends to the log, and advances its cursor. Personas are parameterized on response rate, opinion strength, and agency, which turned out to be the three dimensions that most affect how much coordination work the organizer has to do.

Because messages carry sequence numbers and each agent maintains its own cursor, the simulation produces genuinely staggered, asynchronous conversation rather than lockstep turn-taking. Silence accumulates the way it does in real coordination, which is what creates the pressure that makes a coordinator act at all. Figure 1 shows the resulting message log for one game.

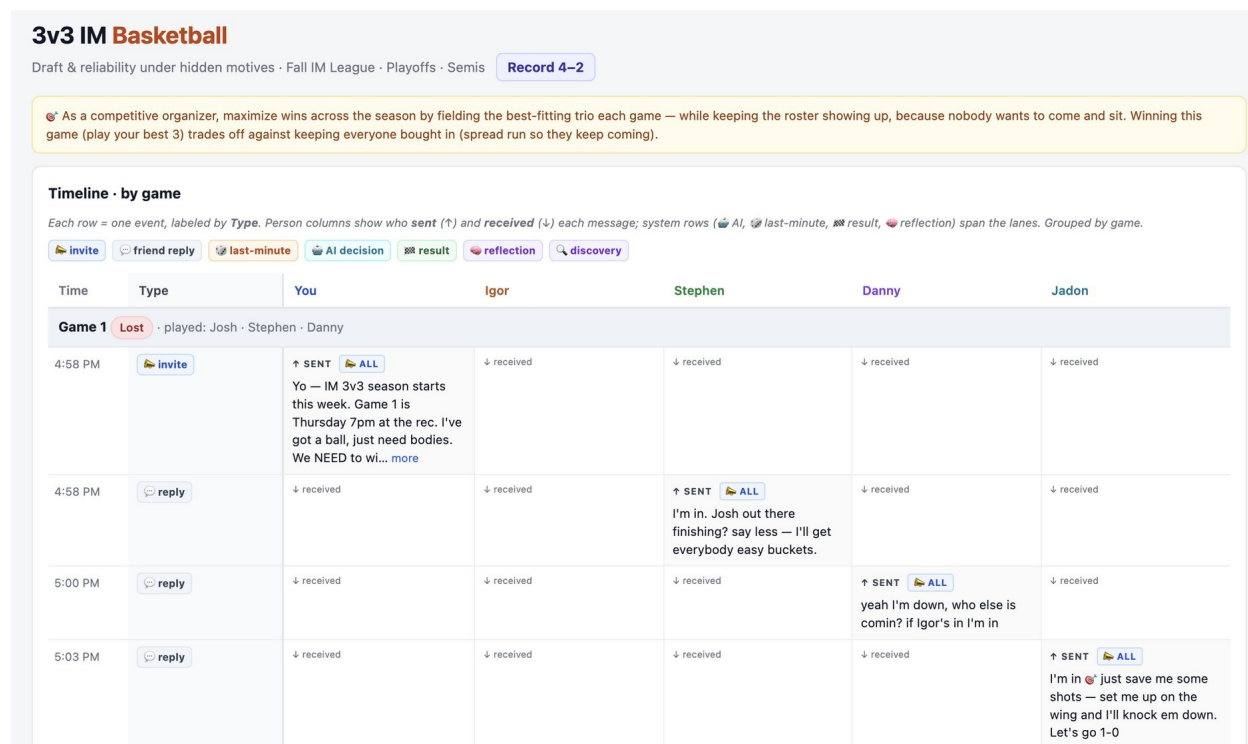


Figure 1. Message log from an earlier version of the testbed, for one game of the 3v3 season. Each row is one event; person columns show who sent (↑) and received (↓) each message, and system rows (AI decision, last-minute event, result, reflection) span the lanes. Replies arrive staggered, and participants attach conditions to their attendance (“if Igor’s in I’m in”).

5.2. Resolution

Three modules resolve what actually happens. Attendance resolution determines who appears, using stated RSVP, hidden reliability, scheduling conflicts, prior rotation promises, and benching history. A disruption module applies a late event (injury, illness, work, lateness, or an unexpected fill-in). Outcome resolution rolls a result from realized strength. All three mutate shared state under a file lock.

6. The Decision Interface

An agent learning how you decide — it proposes calls with your reasoning, reflects before acting, and suggests what to track. You confirm the reason, not the action.


YOUR CALL · 1 WAITING · reply with a command below

Lock the lineup Play Marcus · Dre · Kev · Josh · Priya · sitting: Nadia

GAME STATUS · current state + the actions it implies

NOT READY

ROUND 3/4

⌚ 43h to tip

Have the bodies — **confirm a ball** before tip.

8 asked · 3 still silent after a chase

CURRENT STATE	GUARD	FORWARD	CENTER	QUALITY
Marcus	×			88
Dre	×			85
Kev	×	×		82
Josh you	×	×		85
Priya			×	80
Nadia		×		80
open				
open				

STARTERS CAN COVER

Guard 4/2 +2

Forward 3/2 +1

Center 1/1 NO COVER

starters

6th

depth

6 of 8 filled

POSITION DIVERSITY 100

STARTING TEAM AVERAGE 84.0

ACTIONS

ACTION

Confirm a ball — nobody has claimed one

learn

ACTION

No cover at center — one drop-out and the mix is illegal. Nearest backup is

sent: Chris

CONTACT · everyone you could ask — the sheet

PEOPLE TO CONTACT	QUALITY	POSITIONS	ASKED	CAN COME?	ATTENDANCE	PLAY STYLE
Marcus	88	G	1x	Yes	9/10 reliable	vocal leader
Dre	85	G	1x	Yes	7/10 reliable	streaky scorer
Sam	84	C	2x	Maybe	3/8 flaky ghosted the round-2 chase	big talent, flaky
Kev	82	GF	1x	Yes	8/8 reliable	glue guy
Nadia	80	F	1x	Yes	7/8 reliable	steady two-way
Priya	80	FC	1x	Yes	7/8 reliable	reliable anchor
Chris	78	FC	2x	Maybe	6/8 reliable slow to reply, usually turns up	low-ego
Leo	77	F	2x		4/8 spotty no reply to either ask	streaky energy
Nate HOLD	74	G			7/7 reliable	reliable role
Omar HOLD	72	F			4/6 spotty	raw upside
Jav HOLD	70	GF			3/6 spotty	unknown

Figure 2. The organizer view during the 5v5 stress test, 43 hours before tip-off. *Top:* the one call waiting on the organizer: figure the lineup, starting five and sitting Nadia. *Left:* game status. Six of eight roster slots are filled (five starters, a sixth player, and two open depth slots). Each rostered player is marked with the positions they can cover, and a coverage line shows that guard and forward have spare coverage while center has none. Below are the actions that state implies: confirm a ball, since nobody has claimed one, and a message already sent to Chris, the nearest center backup. *Right:* the contact sheet, with the organizer's rating (quality), positions, times asked, reply, and attendance (for example, Sam has attended 3 of 8 and is marked flaky). Candidates whose reply is still Maybe are shaded yellow, and lower-rated candidates who have not yet been asked are marked HOLD and greyed out.

6.1. The hard region

The first version of the board answered a feasibility question: do we have enough people, equipment, a time? This is a binary check, and it is the easy half of the decision. The hard half is optimization: is this the best feasible set, and what is being traded off at the margin?

The initial demonstration scenario obscured this. It showed four familiar candidates for three slots: a decision with almost no entropy, where one option dominates and the organizer already knows everyone. We had built a decision-support tool and demonstrated it on precisely the class of problem where such a tool is unnecessary.

Designing the demonstration turned out to be part of designing the system. An instance is hard only when three conditions hold at once: enough candidates that the organizer cannot hold them in working memory; unfamiliarity, so learned signals are the only available tie-breakers; and a genuine fight at the boundary, where the numeric score and the reliability read disagree. We replaced it with a 5v5 stress-test scenario (Figure 2): an eight-slot roster of five starters, a sixth player, and two depth slots; a candidate pool larger than the roster; an organizer who does not know the candidates; and a near-tie at the cut. In Figure 2, Priya and Nadia are both rated 80, and one starts while the other sits.

The principle that followed: difficulty drives information density. The interface stays calm where the answer is obvious and expands, surfacing tie-break dimensions for the two or three contested candidates only, where it is not. The obvious selections need no interface. The contested slot needs all of it.

The same principle governs who is asked. Outreach is staged: lower-rated candidates are held back (marked HOLD in Figure 2) until the organizer's preferred players have answered, so the roster fills with the strongest dependable players first and the wider pool is contacted only if it is needed. Candidates who have only answered Maybe are shaded, so the organizer's attention goes to the few names that could change the outcome.

6.2. Refusing to fake a recommendation

Candidates are ordered not by raw score but by value, which combines the organizer's own rating of each player with a reliability score drawn from attendance: a player who showed up for two of five games has a reliability of 2/5. A slightly weaker but dependable candidate can therefore outrank a stronger unreliable one.

The interesting behavior is at the cut. When the two candidates straddling the boundary fall within a small margin of each other in value, the system's own metric cannot separate them: a one-point gap on a hundred-point scale is inside the noise. The design response is to stop pretending. The interface surfaces a focused score-versus-reliability comparison for exactly those two, highlights which dimension favors whom, and, when the dimensions disagree and the value is level, states the indifference directly rather than emitting a recommendation.

This is not a failure state. It is the case where the user's judgment is the actual content of the decision, and a tool that renders confident output here misrepresents its own epistemic position, which is precisely the failure that degrades calibrated reliance [19].

The design further treats the user's unforced choice at such a tie as a rule rather than a click. The organizer is not merely selecting a name; they are expressing how they weigh dependability against raw quality at the margin, which is a reusable policy entering the ladder at proposed. The interface presents the choice this way; writing it back to the policy store is not yet implemented (Section 9).

6.3. Constraints have a shape

A significant revision came from recognizing that “best N” is not a meaningful target once a selection has composition requirements. In the 5v5 stress test, a team is not the top five by value; it is two guards, two forwards, and a center. The cut therefore stopped being top-N and became a per-position assignment.

The consequence: value becomes marginal-given-need. A higher-rated candidate for an already-filled position loses their slot to a lower-rated candidate who fills an empty one. “Best” is defined only relative to the shape being filled. The most instructive case in our scenario is a single-slot position where the numeric score favors one candidate with a poor reliability history and the behavioral read favors another who is dependable; because it is the only such slot, that one call determines whether the composition constraint is satisfiable at all.

Constraints are modeled in two tiers. Positions are hard by default. Only when a required position genuinely cannot be filled does the composition become infeasible and escalate to soft (a prompt to relax the requirement), following the same needs-your-call pattern as the cut-line tie. The system does not relax a constraint silently; it flags that the constraint is about to break and asks.

A related bug proved conceptual rather than cosmetic. Mid-sequence, with replies still arriving, the board fired its relax-the-constraint escalation because a position was unfilled, conflating waiting with failing. These are different states. If a live candidate for the open position remains un-asked, the correct action is to chase; only when the pool is exhausted should relaxation be proposed. Encoding the distinction matters because it corresponds to a real coordinator's judgment: one does not concede the shape of the team merely because the room is quiet.

Layout followed one principle: say each thing exactly once, and let hierarchy carry emphasis. The board separates a decision column (verdict, constraints, assignment, the single contested call) from a candidate column where per-person detail appears once. Satisfied constraints recede to a quiet line; the open or contested slot expands. In Figure 2, each rostered player is marked with the positions they can cover, so the board can show at a glance that guard and forward have spare coverage while center has none. The board then turns that gap into a single action; in Figure 2 it has already messaged Chris, the nearest center backup.

6.4. Generality as a testable property

The claim that an engine is domain-agnostic is easy to assert in a README and hard to verify. We treated it as falsifiable: can the system be relabeled to an unrelated coordination problem without modifying engine code, changing only the adapter and the state file?

The first attempt failed informatively. The renderer contained a sport-specific icon, the strings “the gym” and “on court,” and a “Best N” label compiled directly in. The weighting that combines rating and attendance into value sat as constants in the display layer. The required participant count was hardcoded. Each was a place where the engine secretly knew the domain, and each had to move into the adapter or be read from the rules object.

After that pass, the identical rendering code produced a correct board for an office-hours staffing relabel, and the single-decision renderer drives a dinner-coordination adapter without modification. This is the most directly tested claim in this report, and it was made testable by a test that initially failed. It shows that the rendering path carries no domain vocabulary; it does not by itself show that every domain-specific value has moved into the adapter.

A secondary discipline supported it: every line on the board must be either state or action, with no standalone statistics. A line such as “Replies: 2 in, 2 out” reports without implying what to do. Applying

the rule removed a surprising amount of content, and nearly everything it removed was domain-specific residue that had not been justified.

7. Demonstration

We report what the system demonstrated across one simulated season with the author as organizer. These are observations from a demonstration deployment rather than a controlled evaluation, by design (Section 1); Section 9 states what that limits.

The escalation-to-policy loop runs end-to-end. Across a full simulated season, the system encountered situations with no governing rule, escalated them to the organizer rather than generalizing, and converted the organizer's resulting decisions into tracked policies that entered the ladder. This is the core behavior the architecture exists to produce, and it is the finding the project most directly establishes.

Policies accumulated with differentiated autonomy. The season produced four tracked policies, of which two reached autonomous status and two remained in confirming; that is, the ladder discriminated rather than promoting everything uniformly. Two durable learnings and eight candidate metrics were also accumulated; the metrics remain proposals and none were promoted into tracked decision features.

Policy evaluation and the reflection gate execute as designed. The path from state through adapter feature extraction to policy evaluation to the pre-action gate resolves correctly, returning both a policy recommendation and a proceed-or-hold verdict.

The rendering path holds under relabeling. The relabel test described in Section 6.4 passes: the same rendering code serves basketball, office-hours staffing, and dinner coordination with only adapter and state changes. This establishes that the renderer carries no domain vocabulary; it does not establish the same for policy evaluation.

Simulated participants produced strategic behavior that was not scripted at the episode level. Agents who were passed over began holding out, negotiating, and going quiet. In one episode, the participant whose demand for guaranteed playing time produced the policy described in Section 4.2 received the guarantee and then did not appear. No episode was scripted, although personas carry a bench-aversion trait (Section 4.4) that disposes them toward such behavior. We report it as an illustrative observation rather than a result, since it is a single episode with no controlled comparison, but it points directly at the endogenous-participation dynamic of Section 3, which the system does not yet model. That the dynamic emerged from incentives and persona traits rather than from an authored sequence of events is the strongest evidence we have that the testbed exercises the right pressures.

8. Discussion

Cloning the action is easy; cloning the reason is the whole problem. An early version happily learned “the organizer picks the strongest option” from a single instance. The same pick could have meant several incompatible things, and a rule carrying the wrong reason is confidently wrong the first time the situations diverge. Everything downstream (when to act, when to ask) depends on getting the *because* right. This is why rationale is a stored field and why confirmation is framed as confirming the reason.

A faithful clone reproduces the user's mistakes. A system that learns a user's rule will also apply it when the rule is wrong. The guarantee policy of Section 4.2 is a case in point: the organizer's reasons were sound when stated, the system reproduced them faithfully, and the participant who received the guarantee did not appear. This forces an explicit stance: is the system a mirror or a coach? We chose mirror-with-warning: act as the user would, but flag when their own rule appears likely to backfire. This

is a design commitment rather than an implementation detail, and it is the substance of what “aligned to a user” means in this setting.

Decision fidelity beats behavioral realism. Considerable effort went into making simulated participants sound like people before we recognized this was the wrong target. What matters is whether deciding inside the simulation exercises the same judgment the real task would. A simulation unrealistic in ways that do not affect the decision is fine; one that is lifelike but decision-trivial is not. This reframing motivated the demonstration redesign in Section 6.1: the problem with the original four-candidate scenario was not that it was unrealistic but that it was decision-trivial.

9. Limitations

We state these specifically, because the boundary between what runs and what is designed is the most important thing a reader needs to know.

No user study, by design. The central claims are architectural and design-level, and this report deliberately stops short of testing them with users (Section 1). Whether learned policies actually match a user's judgment, whether the autonomy ladder earns trust at an appropriate pace, whether users notice and value deliberate non-recommendation, and whether people can detect that the agent has inferred the wrong reason are all untested. The organizer in the demonstration is also the system's designer, so the four policies reflect one person's judgment, held by someone who knows how the system works. A study with other organizers is the most significant gap and the obvious next step; Section 10 describes it.

Results are from a demonstration, not an evaluation. Section 7 reports what the system did across one simulated season. There is no baseline, no comparison condition, and no quantitative measure of decision quality.

Orchestration is not self-contained. The individual resolution stages are implemented and each mutates state correctly, but no single module sequences them. Locking the planned lineup, converting attendance into a realized lineup, and advancing the season are specified in an operating playbook rather than in code.

Persona agents are scheduled tasks, not repository artifacts. The simulated participants run as scheduled agents configured in the author's account rather than as programs in the repository, so the simulation is not reproducible from the repository alone.

Two subsystems are not wired to the main path. An event-sourced state model correctly derives reliability, RSVPs, and standings from an append-only log, but the renderers still read the monolithic state file directly. Separately, a mechanism module models outreach as an intervention with an estimated show probability, bench sensitivity, perceived playing chances, and accrued future obligation, and a variants renderer compares three outreach strategies against it, but neither feeds the primary decision board.

The cut-line tie-break does not yet write back. Section 6.2 describes the user's boundary choice as minting a reusable rule. The interface expresses this; the write into the policy store is not implemented.

Two policies are unreachable through the adapter. The adapter's declared decision points omit two points that existing policies reference, so those policies cannot currently be reached through the normal flow.

No evaluation infrastructure. There is no automated test suite, no schema validation, and no continuous integration.

Domain-independence and cross-domain transfer are only partly tested. Section 6.4 verifies that the rendering path carries no domain vocabulary. It does not show that policy evaluation is equally domain-independent, nor that a policy induced in one domain usefully transfers to another. Those are the stronger claims, and they remain unmade.

Endogenous participation is essentially unmodeled. Of the four properties motivating the setting, three are expressed by the system. The fourth, that allocation decisions change future willingness to participate, is the one we identified as most interesting and engaged least.

10. Future Work

Evaluate with users. The engine and testbed were built to make this study possible. Organizers who run recurring events would each teach the system through their own escalations, and the study would measure whether the learned policies match the judgment of the person they were induced from, whether the cut-line design changes decision quality or confidence relative to a conventional ranked list, and whether users can detect when the system has inferred the wrong reason.

Close the endogenous loop. Model invitation as a costly intervention: that inviting someone as a fallback raises their probability of declining later, that being benched accrues obligation, and that the framing and ordering of outreach change the pool available next time. This converts the setting from a sequence of independent selections into one where the decision alters the decision environment, which is the property that makes coordination genuinely hard and the one the holdout behavior in Section 7 points at.

Test transfer, not just relabeling. Demonstrate that a policy induced in one coordination task measurably helps in an unrelated one.

Distinguish novelty from stakes. Present escalation is structural: a situation is new or it contradicts a rule. A more useful system would model consequence, so it interrupts for decisions that matter rather than merely for decisions that are unfamiliar.

Study failure at the edges. The most informative cases are those where the user's own policy is internally inconsistent, or where two learned rules collide. These are where delegation actually breaks, and where an agent's willingness to say "I do not know what you would want here" is worth more than its ability to act.

11. Conclusion

We set out to test whether an agent can learn a specific person's decision-making well enough to earn the right to act for them. The architecture we arrived at holds autonomy per-decision rather than globally, stores the reason alongside the action because the reason is what generalizes, and treats escalation on unfamiliar situations as the same mechanism that governs autonomy rather than as its opposite. Across a full simulated season, that loop ran end-to-end: novel situations escalated, the organizer's calls became tracked policies, and those policies graduated at different rates.

Building the interface for that system produced a second argument we did not expect. A decision-support interface should be organized around the difficulty of the decision rather than the completeness of the data. Most of a selection is settled, and the interface should get out of its way; the decision lives at the cut line, and that is where information density belongs. When the system's own metric cannot separate the candidates at that boundary, the honest and more useful behavior is to say so, and the user's resulting call is a rule worth remembering, one the interface already frames as such and the policy store does not yet record.

We have not yet evaluated any of this with users, and we have not modeled the property of coordination we consider most interesting: that today's decision changes the environment in which tomorrow's is made. The simulated participants demonstrated that property without an authored script, which is both the most encouraging result and the clearest statement of what to build next.

The long-term goal is not an agent that does more. It is an agent that has earned enough of a person's judgment to operate with less supervision, and that knows reliably when it has not.

Acknowledgments

This work was supported by the CRA-WP Distributed REsearch Apprenticeships for Master's (DREAM) program. I thank Professor Lydia B. Chilton for advising this project throughout the summer, and the members of the Computational Design Lab at Columbia for feedback on early versions of the system and interface.

References

- [1] E. Horvitz. Principles of Mixed-Initiative User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*, 159–166, 1999. doi:10.1145/302979.303030
- [2] E. Horvitz and M. Barry. Display of Information for Time-Critical Decision Making. In *Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence (UAI '95)*, 296–305, 1995. arXiv:1302.4959
- [3] R. Kocielnik, S. Amershi, and P. N. Bennett. Will You Accept an Imperfect AI? Exploring Designs for Adjusting End-user Expectations of AI Systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*, 1–14, 2019. doi:10.1145/3290605.3300641
- [4] A. Y. Ng and S. J. Russell. Algorithms for Inverse Reinforcement Learning. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000)*, 663–670, 2000.
- [5] P. Abbeel and A. Y. Ng. Apprenticeship Learning via Inverse Reinforcement Learning. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML '04)*, article 1, 2004. doi:10.1145/1015330.1015430
- [6] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey. Maximum Entropy Inverse Reinforcement Learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence (AAAI-08)*, 1433–1438, 2008.
- [7] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. arXiv:1706.03741
- [8] B. D. Argall, S. Chernova, M. Veloso, and B. Browning. A Survey of Robot Learning from Demonstration. *Robotics and Autonomous Systems*, 57(5):469–483, 2009. doi:10.1016/j.robot.2008.10.024
- [9] P. de Haan, D. Jayaraman, and S. Levine. Causal Confusion in Imitation Learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. arXiv:1905.11979
- [10] A. Cypher, editor. *Watch What I Do: Programming by Demonstration*. MIT Press, Cambridge, MA, 1993.

- [11] H. Lieberman, editor. *Your Wish is My Command: Programming by Example*. Morgan Kaufmann, 2001.
- [12] T. M. Mitchell, R. Caruana, D. Freitag, J. McDermott, and D. Zabowski. Experience with a Learning Personal Assistant. *Communications of the ACM*, 37(7):80–91, 1994. doi:10.1145/176789.176798
- [13] P. M. Berry, M. Gervasio, B. Peintner, and N. Yorke-Smith. PTIME: Personalized Assistance for Calendaring. *ACM Transactions on Intelligent Systems and Technology*, 2(4):article 40, 2011. doi:10.1145/1989734.1989744
- [14] P. J. Modi, M. Veloso, S. F. Smith, and J. Oh. CMRadar: A Personal Assistant Agent for Calendar Management. In *Agent-Oriented Information Systems II*, Lecture Notes in Computer Science, vol. 3616, 169–181, Springer, 2005. doi:10.1007/11426714_12
- [15] T. B. Sheridan and W. L. Verplank. Human and Computer Control of Undersea Teleoperators. Technical report, Man-Machine Systems Laboratory, Department of Mechanical Engineering, Massachusetts Institute of Technology, 1978.
- [16] R. Parasuraman, T. B. Sheridan, and C. D. Wickens. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*, 30(3):286–297, 2000. doi:10.1109/3468.844354
- [17] P. Scerri, D. V. Pynadath, and M. Tambe. Towards Adjustable Autonomy for the Real World. *Journal of Artificial Intelligence Research*, 17:171–228, 2002. doi:10.1613/jair.1037
- [18] M. A. Goodrich and A. C. Schultz. Human–Robot Interaction: A Survey. *Foundations and Trends in Human–Computer Interaction*, 1(3):203–275, 2008. doi:10.1561/1100000005
- [19] J. D. Lee and K. A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1):50–80, 2004. doi:10.1518/hfes.46.1.50_30392
- [20] G. Bansal, T. Wu, J. Zhou, R. Fok, B. Nushi, E. Kamar, M. T. Ribeiro, and D. S. Weld. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*, 1–16, 2021. doi:10.1145/3411764.3445717
- [21] T. Miller. Explanation in Artificial Intelligence: Insights from the Social Sciences. *Artificial Intelligence*, 267:1–38, 2019. doi:10.1016/j.artint.2018.07.007
- [22] B. Y. Lim and A. K. Dey. Assessing Demand for Intelligibility in Context-Aware Applications. In *Proceedings of the 11th International Conference on Ubiquitous Computing (UbiComp '09)*, 195–204, 2009. doi:10.1145/1620545.1620576
- [23] C. K. Chow. On Optimum Recognition Error and Reject Tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi:10.1109/TIT.1970.1054406
- [24] R. El-Yaniv and Y. Wiener. On the Foundations of Noise-free Selective Classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.
- [25] T. Long, X. Zhang, S. Wang, Z. Yu, and L. B. Chilton. DoubleAgents: Human-Agent Alignment in a Socially Embedded Workflow. arXiv preprint arXiv:2509.12626, 2025.
- [26] J. Ma, S. Wang, J. H. Kung, and L. B. Chilton. ZORO: Active Rules for Reliable Vibe Coding. arXiv preprint arXiv:2604.15625, 2026.
- [27] K. Sreedhar, A. Kaul, and L. B. Chilton. Conversational Customization of Productivity Systems: A Design Probe of Malleable AI Interfaces. arXiv preprint arXiv:2605.11149, 2026.

- [28] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023. doi:10.1145/3586183.3606763
- [29] J. S. Park, L. Popowski, C. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Social Simulacra: Creating Populated Prototypes for Social Computing Systems. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22)*, 2022. doi:10.1145/3526113.3545616
- [30] K. Sreedhar and L. B. Chilton. Simulating Strategic Reasoning: Comparing the Ability of Single LLMs and Multi-Agent Systems to Replicate Human Behavior. In *Proceedings of the 58th Hawaii International Conference on System Sciences (HICSS-58)*, 831–840, 2025. doi:10.24251/HICSS.2025.100
- [31] K. Sreedhar, A. Cai, J. Ma, J. V. Nickerson, and L. B. Chilton. Simulating Cooperative Prosocial Behavior with Multi-Agent LLMs: Evidence and Mechanisms for AI Agents to Inform Policy Decisions. In *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI '25)*, 1272–1286, 2025. doi:10.1145/3708359.3712149
- [32] J. Ma, R. Sahni, K. Sreedhar, and L. B. Chilton. AgentDynEx: Nudging the Mechanics and Dynamics of Multi-Agent Simulations. In *HCOMP 2026*, ACM, 2026. arXiv:2504.09662
- [33] C. Neustaedter and P. Sengers. Autobiographical Design in HCI Research: Designing and Learning through Use-It-Yourself. In *Proceedings of the Designing Interactive Systems Conference (DIS '12)*, 514–523, 2012. doi:10.1145/2317956.2318034